Why AI Design Looks Like Slop, and How to Fix It

The tells

Centered hero in Inter. Badge or uppercase pill above the H1. Indigo-to-lavender gradient. Three rounded feature cards with a floating icon and a soft shadow. A colored left-border on cards. A numbered 1-2-3 step row. Low-contrast dark-mode body text. One researcher scored 1,590 real "Show HN" launches against 16 deterministic checks for this exact fingerprint: 22% tripped four or more ("heavy" slop), 32% tripped two or three ("mild"), and even the "clean" 46% mostly had one tell, not zero. This is not a taste complaint — it's a majority of real products wearing the same uniform.

Why it happens — the mechanism, not the vibe

Generation and homogenization are the same phenomenon. A model samples from the highest-probability region of its training distribution. Feed it an under-specified prompt ("modern SaaS landing page") and the highest-probability answer is, definitionally, whatever the training corpus had the most of. This is not a bug you can patch — it's the same mechanism that makes the model useful at all.

The origin is traceable, not mysterious. Tailwind's creator picked `bg-indigo-500` as a neutral demo accent in 2020. It shipped in thousands of tutorials, starter templates, and open-source repos. Models trained on that corpus learned a statistical correlation between "button" and that exact hex value. He publicly apologized for it in 2025 ("leading to every AI generated UI on earth also being indigo"). The mechanism generalizes past color: the centered-hero-plus-three-cards skeleton is the same phenomenon applied to layout, and Inter-by-default is the same phenomenon applied to type.

Alignment training sharpens the effect, deliberately. Pretraining itself is *not* mode-seeking — it actually preserves the long tail. The narrowing happens downstream: RLHF raters measurably prefer familiar, predictable output over novel output (one study estimated this "typicality bias" at $\alpha = 0.57$, $p < 10^{-14}$), which concentrates the model's output distribution onto a smaller set of safe answers than the base model would produce alone.

Uncorrected loops collapse further than a single generation does. A controlled study wired an image model to a captioning model in a closed loop — generate, describe, regenerate, repeat — and ran it 700 times across seven different temperature settings. All 700 runs converged into just twelve generic visual attractors (stock-photo interiors, dramatic landscapes, gothic architecture). Turning up the temperature dial did not prevent it. Diversity is not a free byproduct of randomness; it has to be actively engineered in.

One good screen is not a coherent product. Demos are one high-probability draw, and the center is genuinely competent at one screen. A real product is fifty screens that all have to agree — same spacing rhythm, same meaning for the same blue, same voice in every error message. Each new screen is a fresh, only-weakly-coupled draw from the distribution; as a codebase grows past what fits in the working context, agentic tools lose track of earlier decisions and drift. The tenth screen stops resembling the first not because any single screen is bad, but because nothing is holding the system together between them.

How to make it less like it

The general fix, stated once: stop letting the model *invent* values and make it *select* from a constrained system instead. Every fix below is a version of that same move.

Color. HSL's "lightness" channel is a perceptual lie — pure yellow and pure blue at the same L value look nothing alike, which is why naive lighten/darken functions and default-LLM palettes look chaotic. Use OKLCH instead: it's now the production standard (Tailwind v4 ships its whole palette in it; ~93% browser support). Build ramps with a fixed recipe, not by feel: step lightness evenly top to bottom, let chroma follow an inverted parabola (colors can't hold saturation near white or black), keep hue nearly fixed. Enforce contrast on two rails — WCAG 2's ratio math is the legal gate, but it's a poor perceptual model at the extremes (it fails white-on-orange buttons that read as more legible than the "passing" black-on-orange alternative), so run APCA alongside it as the quality check, not the compliance one. To actually escape indigo: score a candidate brand color's minimum hue-distance against *all* of Tailwind's default families and gate on a real separation, rather than eyeballing it. And fix the muddy-gray-gradient tell directly — it comes from interpolating in RGB, where the midpoint between saturated yellow and saturated blue is literal gray; interpolate in OKLCH instead (`linear-gradient(in oklch, ...)`) and it disappears.

Type. Stop hand-picking sizes; generate every step from one base and one ratio (a modular scale), and let the ratio itself vary with viewport — tighter for dense UI, looser for a marketing hero. Use `clamp()` for fluid sizing, but keep both bounds in `rem` or you silently break browser zoom accessibility. Enforce vertical rhythm: derive a baseline unit from body line-height (16px at 1.5 line-height = 24px) and snap every margin, padding, and line-box to a multiple of it — this is mechanically checkable and most generated pages fail it constantly. Escaping Inter-by-default requires real font metrics (x-height, cap-height, stroke weight pulled from the font file), not adjectives — that's the actual difference between a pairing engine and a guess.

Layout and spacing. Every serious design system converges on a 4px or 8px base spacing unit — Tailwind v4 went further and replaced its whole enumerated scale with a single computed token, making off-grid values a lint error rather than a possibility. Use a real two-dimensional grid (columns, gutters, a max-width), not just stacked flexbox — flexbox is one-dimensional packing logic and is a large part of why generated pages read as a single vertically-stacked column. Density should be a deliberate, enumerated mode (a handful of fixed row heights or a numeric density scale), not padding improvised per screen — and a compact mode needs compensating cues (dividers, zebra striping, aligned numerals) or it's just unreadable. The centered-hero-plus-three-equal-cards pattern is specific enough to catch mechanically: flag any page where every section is a single centered column and any row has three or more siblings with identical widths and matching computed styles, and reject that exact combination in CI.

The system, not the screen. None of the above holds across fifty screens unless it's encoded as *state the model reads from*, not conversation memory. Use a real three-tier token architecture — primitive values (`blue-500`) feed semantic names (`color-action-primary`) feed component-scoped tokens (`button-bg-hover`) — with references flowing one way only. The semantic tier is the actual mechanism: override it once for dark mode or a rebrand and every downstream component updates together; a system that lets components reach past semantics straight to primitives can't be themed at all, and is exactly why generated products lose coherence as they grow. Pair the token file with a queryable index of existing components, so a "primary button" request resolves to your real `<Button>` instead of a freshly invented div — this is the direct fix for the copy-paste/reinvention drift that shows up as near-duplicate components proliferating across a growing codebase.

Fight the convergence directly, and measure it. Under-specification hands the choice back to the model, and the model chooses the median — so specify composition and constraints *before* style: name real hex/OKLCH values, name a reference system, give explicit layout structure, rather than asking for "a modern landing page" and hoping. Where you can condition generation on real reference material — a brand system, curated non-median exemplars — do it; a closed loop with no corrective signal reliably collapses no matter how the sampling parameters are tuned. Don't just assume an intervention worked — measure it: track the Shannon entropy of the accent color and font-family your system actually emits across a batch of generations. If 90 of 100 outputs resolve to the same indigo, the entropy is near zero regardless of what any single output looks like.

A five-minute self-audit

Grep your codebase for `bg-indigo-500` , `#6366f1` , and `#615fff` (Tailwind v3 and v4's literal indigo values) on any primary call-to-action. Check the computed `font-family` on your H1 in devtools — if it resolves to Inter and nobody chose it, that's a default, not a decision. Run a contrast checker against your dark-mode body text specifically; low-contrast dark text is one of the most common tells and a real accessibility failure, not just an aesthetic one. Count how many of the tells at the top of this page your own landing page trips — four or more puts you in the "heavy" bucket, statistically indistinguishable from the median AI output you're trying not to be.